CONCEPTION ORIENTÉE OBJET - UML -

MINI PROJET :

SYSTÈME DE GESTION DE COURS EN LIGNE

Réalisé par:
Nassira Hamich
Supervisé par :
Pr. Anas EL ANSARI

Faculté pluridisciplinaire de Nador
Filière : Informatique et Intelligence Artificielle
Année universitaire : 2025–2026

SOMMAIRE

Introduction 2
Chapitre I - Présentation du Projet 4
Chapitre II- Modélisation UML 6
I- Diagramme de cas d'utilisation : 6
1- Acteurs principaux : 6
2- Principaux cas d'utilisation : 6
3- Relations importantes : 6
4- Diagramme de cas d'utilisation global : 7
II- Diagramme de classes : 7
III- Diagramme d'objets : 20
IV- Diagramme de séquences : 21
V- Diagramme de paquetages : 26
VI- Diagramme de composents : 28
VII- Diagramme d'états-transitions : 29
VIII- Diagramme d'activités : 31
Conclusion 34

INTRODUCTION

L'éducation connaît aujourd'hui une transformation numérique sans précédent. Avec l'essor des technologies de l'information et de la communication, l'apprentissage en ligne s'impose comme une alternative accessible et efficace à l'enseignement traditionnel. Les plateformes d'e-learning se multiplient, offrant à des millions d'apprenants la possibilité d'acquérir de nouvelles compétences, indépendamment des contraintes géographiques et temporelles.

Cependant, malgré cette évolution prometteuse, les plateformes éducatives actuelles présentent plusieurs limites, notamment le manque de contrôle sur les contenus partagés, l'absence de personnalisation intelligente des parcours d'apprentissage, une protection insuffisante des communications, ainsi que la présence envahissante de publicités non éducatives. Ces insuffisances peuvent nuire à la qualité de l'enseignement et à l'expérience des apprenants.

Face à ces défis, notre projet vise à concevoir une **plateforme de gestion de cours en ligne** innovante, combinant des technologies avancées, des mécanismes de sécurité renforcés et des fonctionnalités basées sur l'intelligence artificielle, afin d'offrir une expérience d'apprentissage fiable, personnalisée et sécurisée.

Objectif du projet :

Le présent travail vise à concevoir, à l'aide d'UML (Unified Modeling Language), une plateforme d'apprentissage en ligne qui répond aux besoins actuels en matière d'éducation numérique.

Cette plateforme permettra notamment :

- ✓ La création et la gestion de formations gratuites et payantes
- ✓ L'organisation de groupes et de classrooms sécurisées
- ✓ Le contrôle strict des contenus publiés
- ✓ La surveillance intelligente des communications à l'aide de l'intelligence artificielle
- ✓ La personnalisation de l'expérience d'apprentissage pour chaque utilisateur
- ✓ Le suivi détaillé de la progression de chaque apprenant
- ✓ La protection des droits d'auteur et du travail des formateurs.

Méthodologie :

Notre approche de conception repose sur la **modélisation orientée objet à l'aide d'UML**. Cette méthodologie permet d'identifier clairement les acteurs du système et leurs rôles, de spécifier les besoins fonctionnels et non fonctionnels, et de concevoir l'architecture globale du système à travers différents diagrammes UML. Elle garantit également la cohérence, la lisibilité et la traçabilité de la conception tout au long du processus de développement.

Structure du document :

Ce rapport est organisé en deux chapitres principaux :

Chapitre I : présente le contexte général du projet, analyse les problématiques actuelles de l'e-learning .Il définit les objectifs et fonctionnalités de la plateforme proposée, identifie les acteurs du système et décrit en détail les exigences fonctionnelles et non fonctionnelles.

Chapitre II: présente l'ensemble des diagrammes UML nécessaires à la conception du système (diagrammes de cas d'utilisation, de classes,d'objets, de séquences, d'activités, d'états et de composants…).

Enfin, une conclusion générale synthétisera les apports du projet.

CHAPITRE I - PRÉSENTATION DU PROJET

1- Contexte et problématique

L'éducation en ligne a connu une croissance exponentielle ces dernières années, accélérée par la pandémie de COVID-19. Les plateformes d'**E-learning** offrent aujourd'hui accessibilité et flexibilité à des millions d'apprenants à travers le monde.

Cependant, cette démocratisation révèle des failles critiques : absence de contrôle sur les contenus partagés, communications non sécurisées exposant les utilisateurs au harcèlement et *au spam*, parcours d'apprentissage standardisés sans personnalisation, et protection insuffisante des contenus payants face au piratage. Ces problèmes nuisent directement à la qualité pédagogique et à l'expérience utilisateur.

Notre proposition : Une plateforme d'apprentissage intégrant l'intelligence artificielle à tous les niveaux pour garantir un environnement exclusivement éducatif, sécurisé et personnalisé.

2. Acteurs et rôles :

Acteur	Description et rôle
Apprenant	Utilisateur principal qui consulte des cours, passe des quiz, suit sa progression et communique avec les utilisateurs.
Formateur	Crée et gère les formations (gratuites ou payantes), organise le contenu en modules, suit la progression des apprenants et contrôle les accès.
Administrateur	Supervise la plateforme, gère les utilisateurs, traite les infractions signalées et assure le bon fonctionnement du système.
Système d'IA	Acteur automatique qui analyse les contenus publiés, surveille les communications, personnalise l'apprentissage.

3. Les fonctionnalités principales :

- **Gestion des formations :** Création de formations gratuites/payantes avec organisation modulaire, upload de ressources multimédias (vidéo, PDF, audio, image…), codes d'accès sécurisés et paiement intégré.
- **Groupes et Classrooms sécurisées :** Espaces collaboratifs privés protégés par code d'accès, avec gestion des permissions et suivi en temps réel des activités.
- **Intelligence Artificielle intégrée :**

- ✓ **Contrôle des contenus** : Analyse automatique de tous les textes, images, vidéos et liens avant publication. Filtrage des contenus non éducatifs.
- ✓ **Personnalisation** : Recommandations de cours adaptées au profil, quiz générés selon le niveau de l'apprenant, résumés automatiques des vidéos et documents.
- ✓ **Surveillance des communications** : Détection automatique du harcèlement, spam et contacts personnels dans le chat privé, sans lecture humaine.
- ✓ **Suivi pédagogique** : Identification précoce des difficultés, alertes sur les concepts non maîtrisés.

- **Système de progression :** Tableau de bord avec statistiques détaillées (vidéos visionnées, PDF consultés, quiz réussis), visualisation graphique de l'évolution, identification automatique des points forts et faiblesses, feedback détaillé par question.
- **Gestion des infractions :** Enregistrement automatique par l'IA avec avertissements progressifs (avertissement → suspension → bannissement). Liste consultable uniquement par l'administrateur, sans accès aux contenus privés.

4- Exigences non fonctionnelles :

- **Sécurité:**authentification sécurisée, chiffrement des données, protection contre les attaques.
- **Performance :** temps de réponse rapide(< 2s), haute disponibilité (99.9%), scalabilité pour croissance utilisateurs.
- **Confidentialité :** Conformité RGPD, protection des données personnelles, transparence totale.
- **Utilisabilité :** Interface intuitive, responsive (mobile/tablette/desktop), multilingue, accessible (normes WCAG).

CHAPITRE II- MODÉLISATION UML

I- Diagramme de cas d'utilisation :

Le diagramme de cas d'utilisation représente l'ensemble des interactions entre les acteurs et le système de gestion de cours en ligne. Il présente les fonctionnalités principales accessibles à chaque type d'utilisateur ainsi que l'intégration du Système IA pour garantir la sécurité et la qualité des contenus.

1- Acteurs principaux :

- ✓ **Apprenant** -Utilisateur de base du système
- ✓ **Formateur** -Hérite des fonctionnalités de l'Apprenant
- ✓ **Administrateur** -Hérite des fonctionnalités de l'Apprenant
- ✓ **Système IA** - Acteur automatique

2- Principaux cas d'utilisation :

Pour l'Apprenant : L'apprenant peut s'inscrire et se connecter au système pour accéder aux fonctionnalités d'apprentissage. Il peut rejoindre une formation gratuite ou payante selon ses besoins. Une fois inscrit, il a la possibilité de publier du contenu, de suivre sa progression dans les cours, de participer au chat privé ou public pour communiquer avec les autres utilisateurs, et de bénéficier de l'assistance IA pour obtenir des réponses à ses questions.

Pour le Formateur : Le formateur dispose de capacités supplémentaires pour créer une formation et gérer les ressources pédagogiques associées. Grâce à l'héritage, le formateur bénéficie également de toutes les fonctionnalités de l'apprenant, ce qui lui permet de s'inscrire, se connecter et participer aux formations en tant qu'apprenant si nécessaire.

Pour l'Administrateur : L'administrateur supervise le système en gérant les utilisateurs et en consultant la liste de surveillance pour identifier les comportements suspects. Il est responsable de gérer les infractions détectées et de modérer les contenus inappropriés. Comme le formateur, l'administrateur hérite de toutes les fonctionnalités de l'apprenant.

Pour le Système IA : Le système IA intervient automatiquement pour analyser les publications incluant les liens et le contenu textuel afin de détecter d'éventuelles violations. Il surveille les chats en temps réel pour maintenir un environnement sûr et respectueux. De plus, il génère des recommandations personnalisées basées sur le comportement et les préférences des apprenants, et répond automatiquement aux requêtes des utilisateurs via l'assistance IA.

3- Relations importantes :

Le diagramme utilise plusieurs types de relations UML. Les relations **<<include>>** indiquent des dépendances **obligatoires** : la formation payante inclut le paiement, publier du contenu inclut l'analyse par le Système IA, et cette analyse inclut la modération si une infraction est détectée. Le chat privé/public inclut la surveillance des chats par le Système IA, et l'assistance IA inclut la réponse aux requêtes utilisateur.

Les relations de **généralisation** (héritage) montrent que la formation gratuite et la formation payante sont deux variantes de "Rejoindre des formations". Le Formateur et l'Administrateur héritent des fonctionnalités de l'Apprenant, ce qui leur permet d'accéder aux mêmes fonctionnalités de base tout en disposant de leurs propres capacités spécifiques.

4- Diagramme de cas d'utilisation global :

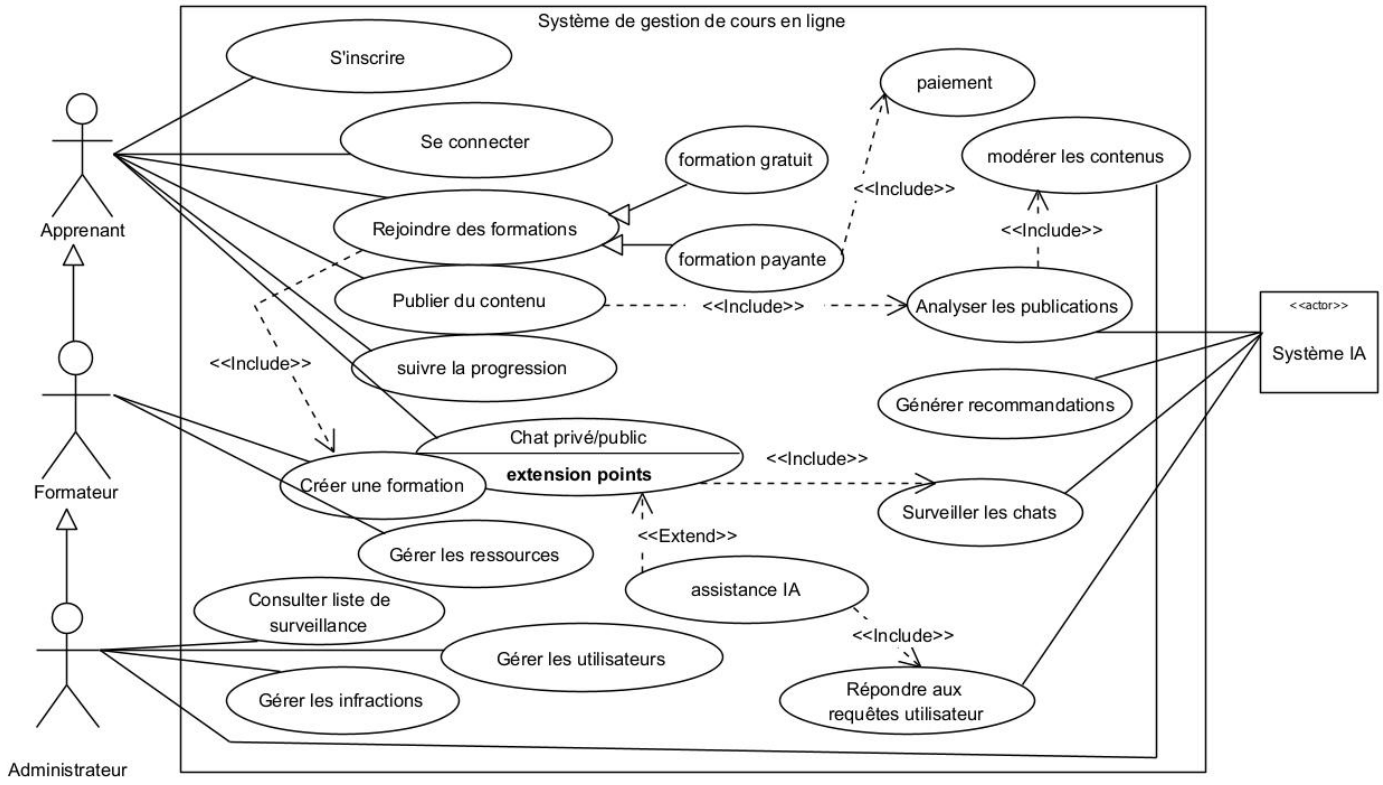


Figure 1 : Diagramme de cas d'utilisation global

II- Diagramme de classes :

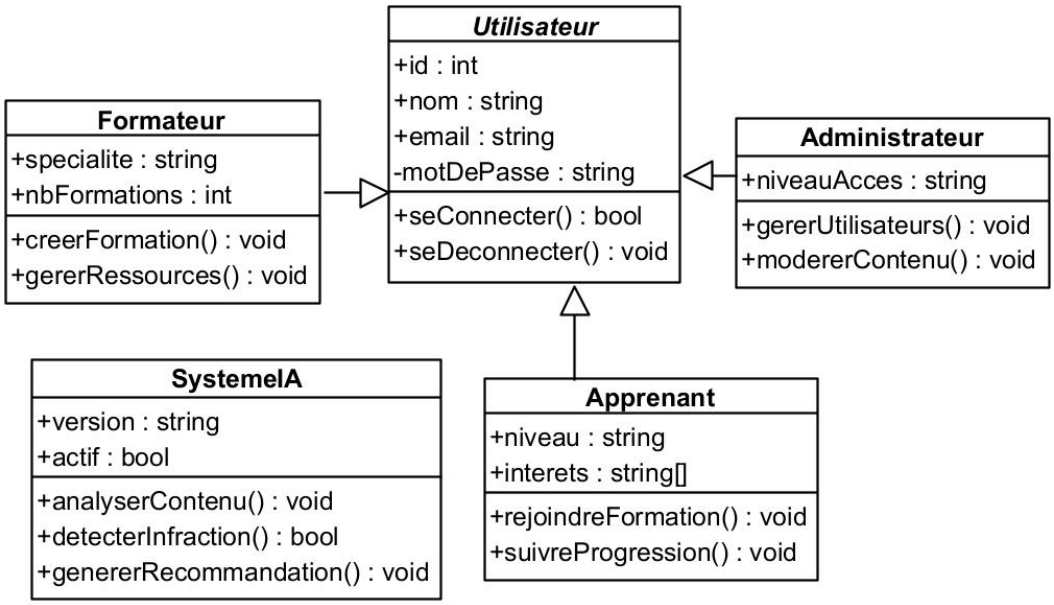
Le diagramme de classes représente la structure statique du système en décrivant les classes, leurs attributs, leurs opérations et les relations entre elles. Il constitue le cœur de la modélisation orientée objet.

1- Classes principales et leur représentation :

Les classes du système sont organisées en six **domaines fonctionnels** :
Domaine Utilisateur : Gestion des comptes et profils

- **Utilisateur** (classe abstraite) : Classe mère de tous les utilisateurs avec les attributs communs (id, nom, email, motDePasse) et les méthodes de base (seConnecter(), seDeconnecter())
- **Apprenant**: Hérite de Utilisateur et dispose de méthodes spécifiques (rejoindreFormation(), suivreProgression())
- **Formateur**: Hérite de Utilisateur avec des capacités supplémentaires (creerFormation(), gererRessources())
- **Administrateur** : Hérite de Utilisateur et supervise le système (gererUtilisateurs(), modererContenu())

Le Diagramme de classe :



Le code C++ pour chaque classe:

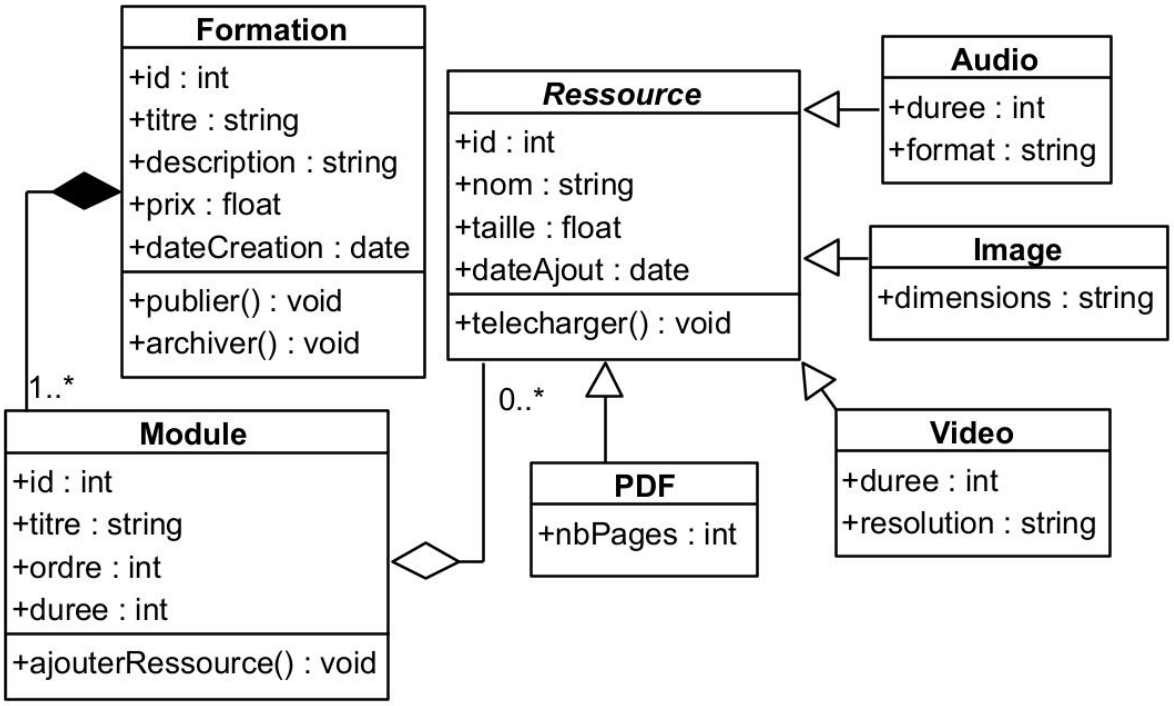
```
// Classe Utilisateur (classe de base)
class Utilisateur {
      protected:
            int id;
            std::string nom;
            std::string email;
            std::string motDePasse;
      public:
            Utilisateur(int id, const std::string& nom, const std::string&
      email, const std::string& motDePasse);
            virtual ~Utilisateur() = default;
            bool seConnecter(); void seDeconnecter(); };
// Classe Apprenant
class Apprenant : public Utilisateur {
    private:
      std::string niveau;
      std::string interets[];
```

```
 public:
      Apprenant(int id, const std::string& nom, const std::string& email,
const std::string& motDePasse, const std::string& niveau);
      void rejoindreFormation();
      void suivreProgression(); };
// Classe Formateur
class Formateur : public Utilisateur {
      private:
            std::string specialite;
            int nbFormations;
      public:
            Formateur(int id, const std::string& nom, const std::string&
      email, const std::string& motDePasse, const std::string& specialite);
            void creerFormation();
            void gererRessources(); };
// Classe Administrateur
class Administrateur : public Utilisateur {
      private:
            std::string niveauAcces;
      public:
            Administrateur(int id, const std::string& nom, const
      std::string& email, const std::string& motDePasse, const
      std::string& niveauAcces);
            void gererUtilisateurs();
            void modererContenu(); };
// Classe SystemeIA
class SystemeIA {
      private:
            std::string version;
            bool actif;
      public:
            SystemeIA(const std::string& version);
            void analyserContenu();
            bool detecterInfraction();
            void genererRecommandation(); };
```

Domaine Formation : Gestion des cours et contenus

- **Formation** : Représente un cours complet avec titre, description, prix et dateCreation
- **Module** : Sous-partie d'une formation permettant d'organiser le contenu
- **Ressource** : Classe abstraite pour tout contenu pédagogique (nom, taille, dateAjout)
- **Video, PDF, Image, Audio** : Héritent de Ressource avec leurs attributs spécifiques

Le Diagramme de classe :



Le code C++ pour chaque classe:

```
// Classe Formation
class Formation {
      private:
            int id;
            std::string titre;
            std::string description;
            float prix; std::tm dateCreation;
      public:
            Formation(int id, const std::string& titre, const std::string&
      description, float prix, const std::tm& dateCreation);
            void publier();
            void archiver(); };
// Classe Module
class Module {
      private:
            int id;
            std::string titre;
            int ordre;
            int duree;
      public:
            Module(int id, const std::string& titre, int ordre, int duree);
      void ajouterRessource(); };
// Classe Ressource (classe de base abstraite)
class Ressource {
```

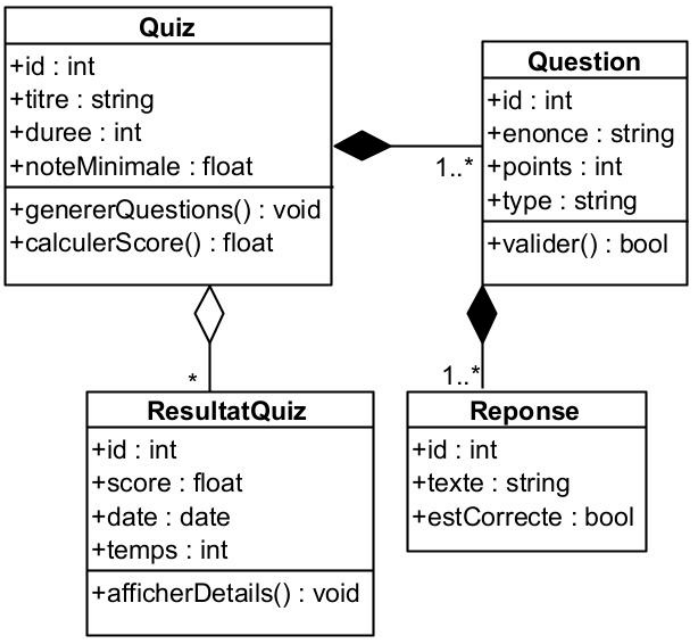
```
	protected:
		int id;
		std::string nom;
		float taille;
		std::tm dateAjout;
	public:
		Ressource(int id, const std::string& nom, float taille, const
	std::tm& dateAjout);
		virtual ~Ressource() = default; void telecharger(); };
// Classe PDF
class PDF : public Ressource {
	private:
		int nbPages;
	public:
		PDF(int id, const std::string& nom, float taille, const
	std::tm& dateAjout, int nbPages); };
// Classe Video
class Video : public Ressource {
	private:
		int duree;
		std::string resolution;
	public:
		Video(int id, const std::string& nom, float taille, const
	std::tm& dateAjout, int duree, const std::string& resolution); };
// Classe Audio
class Audio : public Ressource {
	private:
		int duree;
		std::string format;
	public:
		Audio(int id, const std::string& nom, float taille, const
	std::tm& dateAjout, int duree, const std::string& format); };
// Classe Image
class Image : public Ressource {
	private:
		std::string dimensions;
	public:
		Image(int id, const std::string& nom, float taille, const
	std::tm& dateAjout, const std::string& dimensions); };
```

Domaine Évaluation : Gestion des quiz et résultats

- **Quiz** : Évaluation automatique avec titre, durée et note minimale
- **Question** : Élément d'un quiz avec son énoncé et ses points
- **Reponse** : Réponse possible à une question avec indicateur de correction

- **ResultatQuiz** : Enregistre les résultats d'un apprenant (score, date, temps)

Le Diagramme de classe :



Le code C++ pour chaque classe :

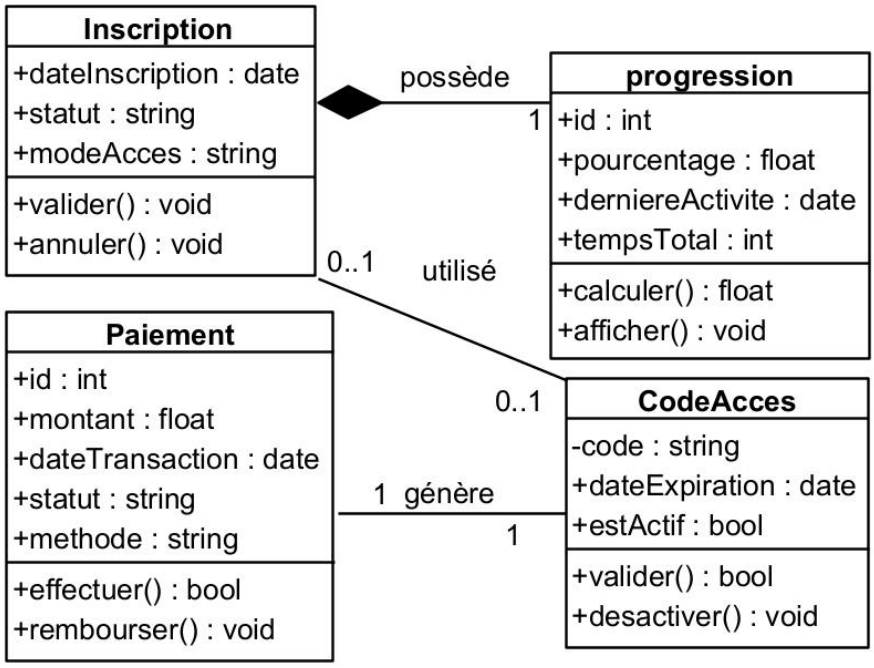
```
// Classe Quiz
class Quiz {
      private:
            int id;
            std::string titre;
            int duree;
            float noteMinimale;
      public:
            Quiz(int id, const std::string& titre, int duree, float
      noteMinimale);
            void genererQuestions();
            float calculerScore(); };
// Classe Question
class Question {
      private:
            int id;
            std::string enonce;
            int points;
            std::string type;
      public:
            Question(int id, const std::string& enonce, int points, const
      std::string& type);
            bool valider(); };
// Classe Reponse
```

```
class Reponse {
	private:
		int id;
		std::string texte;
		bool estCorrecte;
	public:
		Reponse(int id, const std::string& texte, bool estCorrecte); };
// Classe ResultatQuiz
class ResultatQuiz {
	private:
		int id;
		float score;
		std::tm date;
		int temps;
	public:
		ResultatQuiz(int id, float score, const std::tm& date, int
	temps);
		void afficherDetails(); };
```

Domaine Accès et Paiement : Gestion des inscriptions

- **CodeAcces** : Code unique pour accéder à une formation (code, dateExpiration, estActif)
- **Paiement** : Transaction financière (montant, dateTransaction, statut, methode)
- **Inscription** : Lien entre apprenant et formation (dateInscription, statut)
- **Progression** : Suivi de l'avancement d'un apprenant (pourcentage, derniereActivite)

Le diagramme de classe :



Le code C++ pour chaque classe:

classe Inscription :

```
// Classe Inscription
class Inscription {
      private:
            std::tm dateInscription;
            std::string statut;
            std::string modeAcces;
      public:
            Inscription(const std::tm& dateInscription, const std::string&
      statut, const std::string& modeAcces);
            void valider();
            void annuler(); };
// Classe Progression
class Progression {
      private:
            int id;
            float pourcentage;
            std::tm derniereActivite;
            int tempsTotal;
      public:
            Progression(int id, float pourcentage, const std::tm&
      derniereActivite, int tempsTotal);
            float calculer();
            void afficher(); };
// Classe CodeAcces
class CodeAcces {
      private:
            std::string code; std::tm dateExpiration;
            bool estActif;
      public:
            CodeAcces(const std::string& code, const std::tm&
      dateExpiration, bool estActif);
            bool valider(); void desactiver(); };
// Classe Paiement
class Paiement {
      private:
            int id;
            float montant;
            std::tm dateTransaction;
            std::string statut;
            std::string methode;
      public:
```

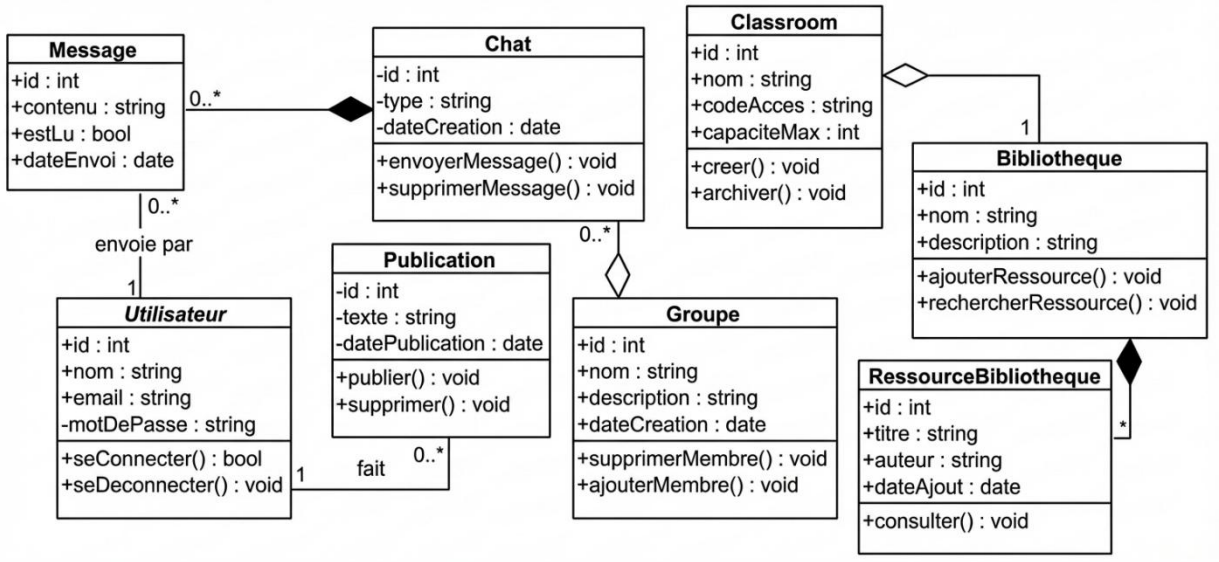
```
        Paiement(int id, float montant, const std::tm& dateTransaction,
    const std::string& statut, const std::string& methode);
        bool effectuer(); void rembourser(); };
```

Domaine Communication : Gestion des interactions

- **Chat** : Conversation entre utilisateurs (type, dateCreation)
- **Message** : Contenu d'une communication (contenu, dateEnvoi, estLu)
- **Publication** : Contenu posté par un utilisateur (texte, datePublication)
- **Groupe** : Ensemble d'apprenants (nom, description, dateCreation)
- **Classroom** : Espace de travail sécurisé (nom, codeAcces, capaciteMax)
- **Bibliotheque** : Espace de ressources partagées servant de stockage centralisé pour les documents pédagogiques (nom, description)
- **RessourceBibliotheque** : Document dans la bibliothèque (titre, auteur, dateAjout)

Note : *La bibliothèque représente la structure de données pour le stockage et l'organisation des ressources, accessibles via les formations et modules.*

Le diagramme de classe :



Le code C++ pour chaque classe:

```
// Classe Message
class Message {
    private:
        int id;
        std::string contenu;
        bool estLu; std::tm dateEnvoi;
    public:
        Message(int id, const std::string& contenu, bool estLu, const
    std::tm& dateEnvoi); };
// Classe Chat
class Chat {
```

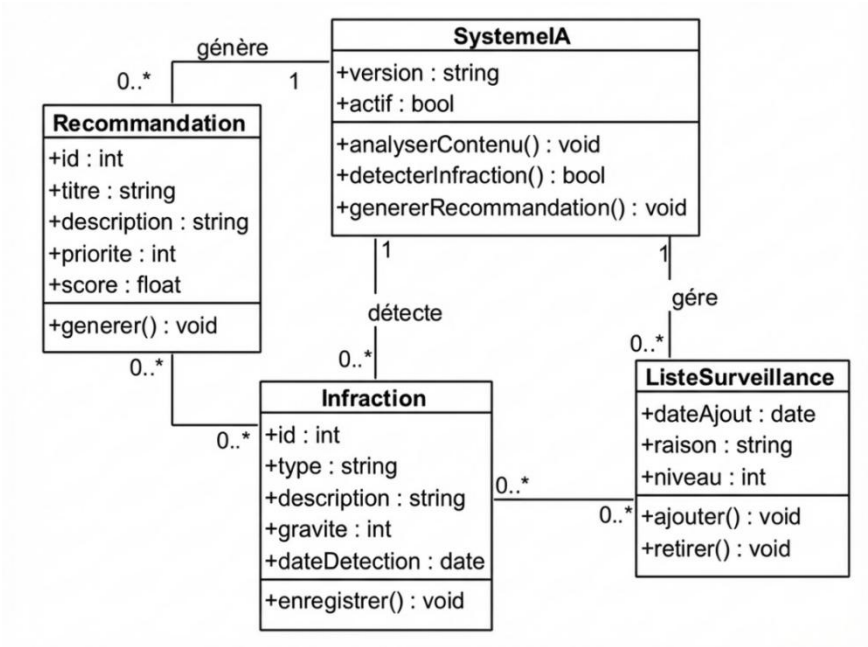
```
        private:
                int id;
                std::string type;
                std::tm dateCreation;
        public:
                Chat(int id, const std::string& type, const std::tm&
        dateCreation);
                void envoyerMessage();
                void supprimerMessage(); };
// Classe Groupe
class Groupe {
        private:
                int id;
                std::string nom;
                std::string description;
                std::tm dateCreation;
        public:
                Groupe(int id, const std::string& nom, const std::string&
        description, const std::tm& dateCreation);
                void supprimerMembre(); void ajouterMembre(); };
// Classe Publication
class Publication {
        private:
                int id;
                std::string texte;
                std::tm datePublication;
        public:
                Publication(int id, const std::string& texte, const std::tm&
        datePublication);
                void publier(); void supprimer(); };
// Classe Classroom
class Classroom {
        private:
                int id;
                std::string nom;
                std::string codeAcces;
                int capaciteMax;
        public:
                Classroom(int id, const std::string& nom, const std::string&
        codeAcces, int capaciteMax);
                void creer();
                void archiver(); };
// Classe Bibliotheque
class Bibliotheque {
```

```
    private:
        int id;
        std::string nom;
        std::string description;
    public:
        Bibliotheque(int id, const std::string& nom, const std::string&
    description);
        void ajouterRessource();
        void rechercherRessource(); };
// Classe RessourceBibliotheque
class RessourceBibliotheque {
    private:
        int id;
        std::string titre;
        std::string auteur;
        std::tm dateAjout;
    public:
        RessourceBibliotheque(int id, const std::string& titre, const
    std::string& auteur, const std::tm& dateAjout);
        void consulter(); };
```

Domaine Sécurité : Gestion de la surveillance et infractions

- **SystemeIA** : Gère toutes les analyses automatiques (analyserContenu(), detecterInfraction(), genererRecommandation())
- **Infraction** : Enregistre une violation des règles (type, description, gravite, dateDetection)
- **ListeSurveillance** : Gestion des utilisateurs à risque (dateAjout, raison, niveau)
- **Recommandation** : Suggestion générée par l'IA (titre, description, priorite)

Le Diagramme de classe :



Le code C++ pour chaque classe :

```
// Classe SystemeIA
class SystemeIA {
private:
            std::string version;
            bool actif;
      public:
            SystemeIA(const std::string& version, bool actif);
            void analyserContenu();
            bool detecterInfraction();
            void genererRecommandation(); };
// Classe Recommandation
class Recommandation {
      private:
            int id;
            std::string titre;
            std::string description;
            int priorite;
            float score;
      public:
            Recommandation(int id, const std::string& titre, const
      std::string& description, int priorite, float score);
            void generer(); };
// Classe Infraction
class Infraction {
      private:
            int id;
            std::string type;
            std::string description;
            int gravite;
            std::tm dateDetection;
      public:
            Infraction(int id, const std::string& type, const std::string&
      description, int gravite, const std::tm& dateDetection);
            void enregistrer(); };
// Classe ListeSurveillance
class ListeSurveillance {
      private: std::tm dateAjout;
            std::string raison;
            int niveau;
      public:
            ListeSurveillance(const std::tm& dateAjout, const std::string&
      raison, int niveau);
            void ajouter(); void retirer(); };
```

2- Relations principales :

Le diagramme utilise plusieurs types de relations pour modéliser les interactions entre les classes :

Relations d'héritage : Les classes Apprenant, Formateur et Administrateur héritent de la classe abstraite Utilisateur, partageant ainsi les attributs et méthodes communes. De même, les classes Video, PDF, Image et Audio héritent de la classe abstraite Ressource.

Associations principaux:

- Un Apprenant peut s'inscrire à plusieurs Formations via la classe Inscription (1..*)
- Une Formation est composée de plusieurs Modules (1..* composition)
- Un Module contient plusieurs Ressources (0..* agrégation)
- Un Quiz contient un ou plusieurs Questions (1..* composition)
- Une Question possède plusieurs Reponses possibles (1..* composition)
- Un Apprenant peut avoir plusieurs ResultatQuiz (0..*)
- Un Chat regroupe plusieurs Messages (1..* composition)
- Le SystemeIA analyse les Publications et détecte les Infractions (associations de dépendance)
- Un Apprenant suit sa Progression dans chaque Formation (1..1 association)
- Le SystemeIA génère des Recommandations pour les Apprenants (0..* association)

Compositions et agrégations : Les relations de composition indiquent qu'une partie ne peut exister sans le tout (Quiz-Question, Chat-Message). Les agrégations représentent des relations plus souples où les éléments peuvent exister indépendamment (Module-Ressource).

3- Diagramme de classes complet :

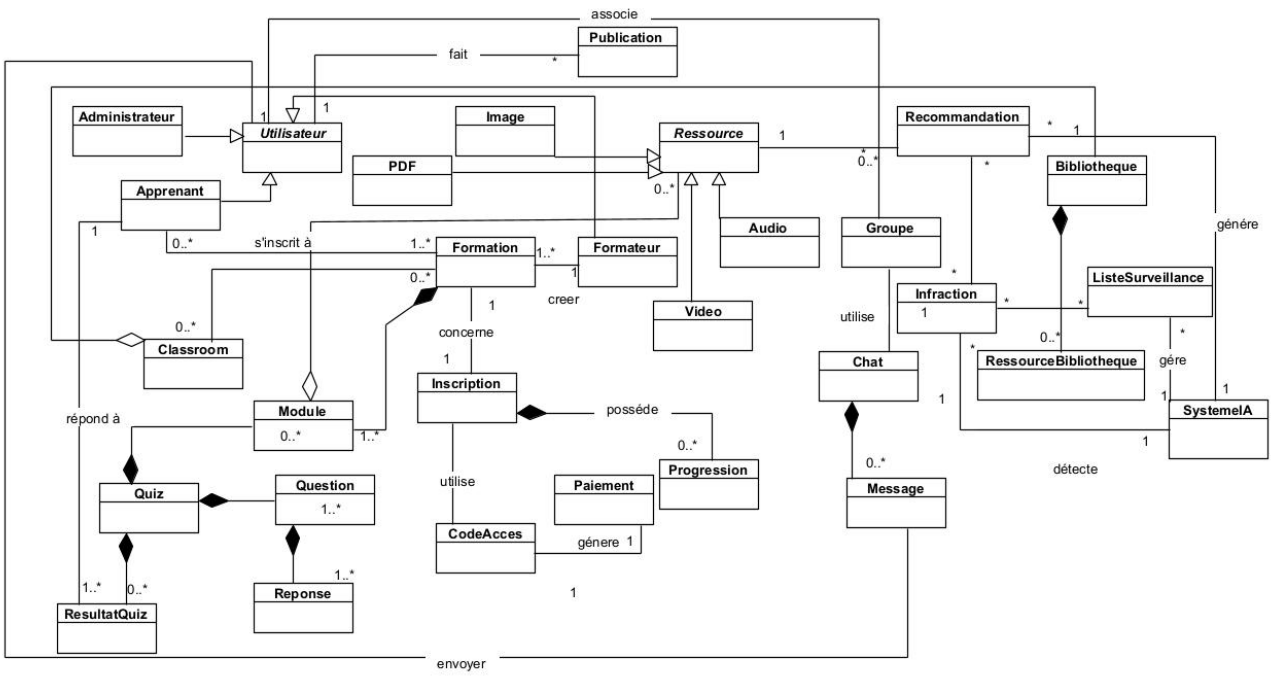


Figure 2 : Diagramme de classes complet

III- Diagramme d'objets :

Le diagramme d'objets représente une instance du système à un moment donné. Il illustre des objets concrets avec leurs valeurs d'attributs et les liens entre eux. Le diagramme d'objets capture un instantané du système en fonctionnement.

♦ *Exemple d'instance 1 : "Apprenant suivant une formation":*

Le diagramme ci-dessous montre un cas concret où l'apprenant Anonymous User (identifiant 11001) est inscrit à la formation "Python pour Débutants" créée par le formateur Dr. xxxx - xxxx (identifiant 2446). L'apprenant a complété 65% de la formation et a passé un quiz avec un score de 85%. Le système d'intelligence artificielle a analysé sa progression et lui a recommandé une formation avancée "Python avancé" avec un score de pertinence de 0.92.

♦ *Objets représentés :*

Cet exemple illustre les interactions typiques dans le système : apprenant suit une formation, sa progression est suivie automatiquement, il passe des quiz qui sont corrigés instantanément, et l'IA génère des recommandations personnalisées basées sur ses performances.

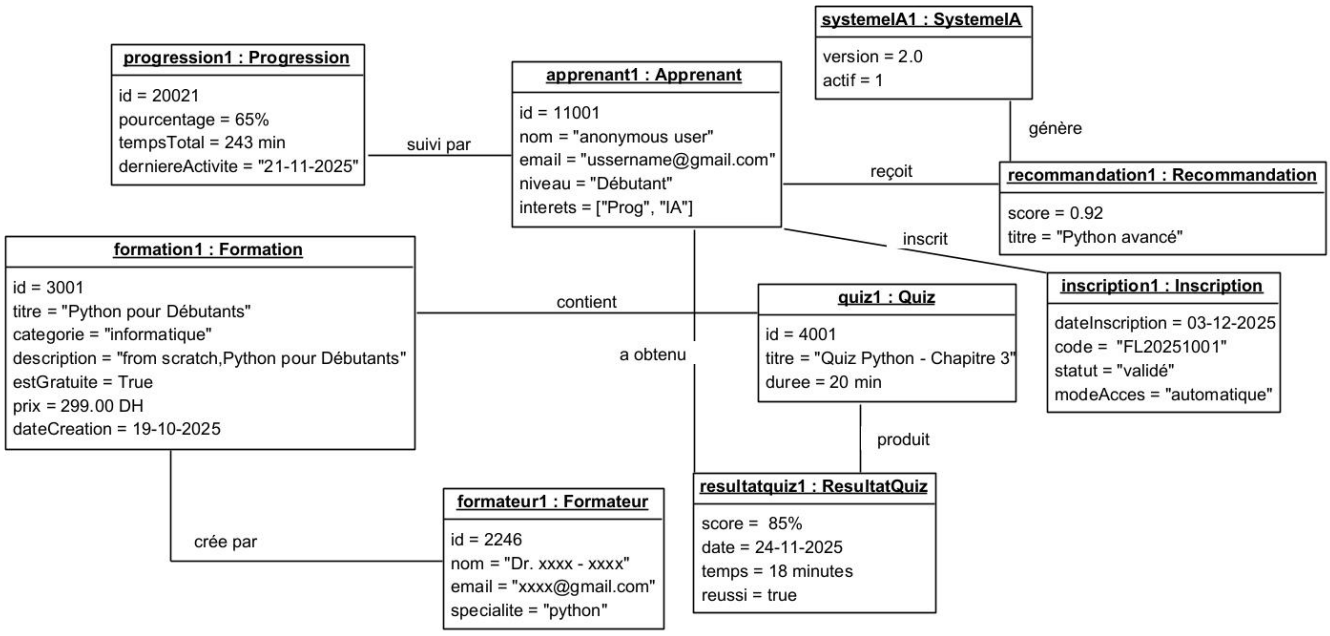


Figure 3 : Diagramme d'objets - Instance concrète du système

IV- Diagramme de séquences :

Les diagrammes de séquence représentent les interactions entre objets dans le temps. Nous présentons ici trois scénarios essentiels du système.

1- Scénario 1 : Publier du contenu (filtré par IA) .

Contexte : Un utilisateur souhaite partager un contenu pédagogique (texte, images, vidéos) avec la communauté. Le système doit vérifier que ce contenu respecte les règles de la plateforme avant de le publier.

Acteurs impliqués :

- **Utilisateur** :Créateur du contenu
- **Interface** : Interface de publication
- **Système** : Le système central qui coordonne les opérations
- **SystemeIA** : Analyseur de contenu
- **BaseDeDonnees** : Le système de stockage

♦ Déroulement chronologique détaillé :

1. **Saisie du contenu :** L'utilisateur accède à l'interface de publication et saisit son contenu (texte, images, liens) et clique sur «Publier».
2. **Transmission au système :** L'interface envoie le contenu au système central .
3. **Appel de l'IA :** Le système central invoque le SystemeIA pour analyser le contenu.
4. **. Analyse par l'IA :** Le SystemeIA effectue:
    - **Analyse sémantique du texte .**
    - **Vérification des images.**
    - **Validation des vidéos .**
    - **Détection de plagiat.**

Décision de l'IA : le SystemeIA retourne une décision binaire : "approuvé" si le contenu est jugé éducatif et conforme, "refusé" dans le cas contraire. Cette décision est accompagnée d'un niveau de confiance et, en cas de refus, d'une explication.

Si approuvé :

- Le système enregistre la publication dans la base.
- Message de succès affiché à l'utilisateur.

Si refusé :

- Le SystemeIA enregistre l'infraction
- Message d'erreur avec explication affiché
- Compteur d'infractions incrémenté

Diagramme de séquence :

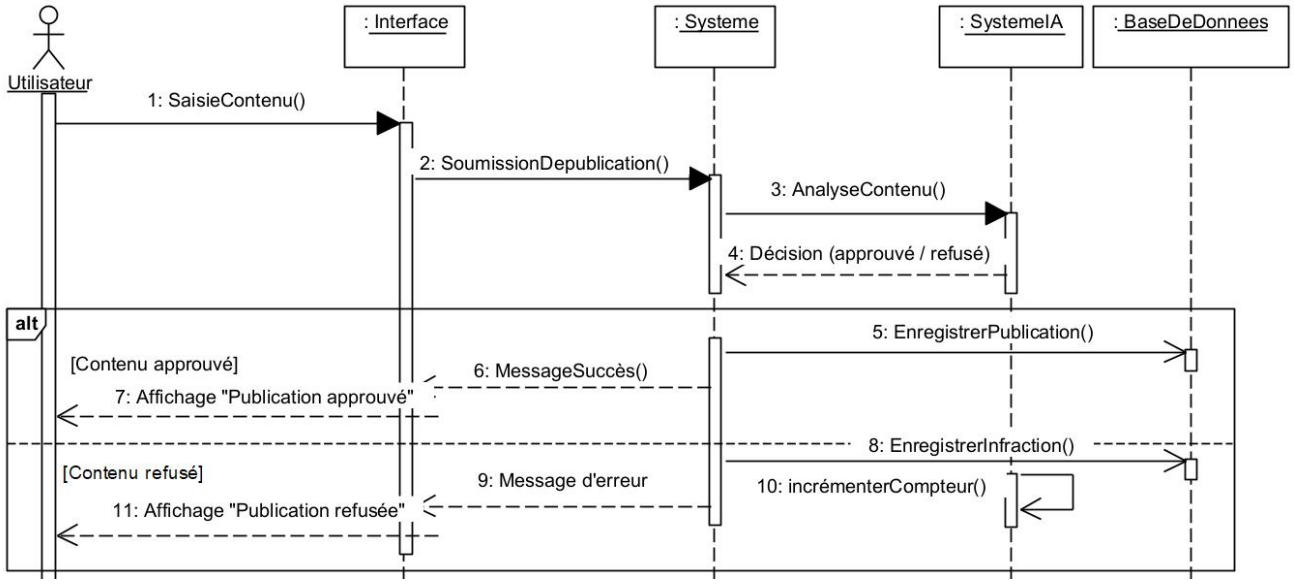


Figure 4 : Diagramme de séquence - Publier du contenu avec validation IA

Note relative à la modélisation : Bien que Visual Paradigm affiche une barre d'exécution sur l'acteur, celle-ci ne signifie pas qu'il effectue un traitement.Conformément aux standards UML, l'acteur est ici un intervenant passif ; la barre ne sert qu'à marquer sa participation à l'échange.

2- Scénario 2 : Chat privé surveillé par IA .

Contexte : Deux utilisateurs souhaitent communiquer via le chat. Le système IA surveille les messages en temps réel pour détecter les contenus inappropriés.
Principe de confidentialité : Les messages ne sont **JAMAIS** lus par un administrateur humain. Seule l'IA effectue une analyse automatique, et les administrateurs ne reçoivent que des alertes anonymisées en cas de problème.
Acteurs impliqués :

- **Expediteur** : L'utilisateur qui envoie le message

- **Interface** : L'interface de chat
- **Système** : Le système de messagerie
- **SystemeIA** : L'analyseur de messages
- **Destinataire** : L'utilisateur qui reçoit le message

♦ Déroulement chronologique détaillé :

1. **Sélection du destinataire :** L'expéditeur sélectionne un contact ,tape son message et lique sur «Envoyer».
2. **Transmission :** L'interface envoie le message au système.
3. **Analyse en temps réel :** Le système invoque le SystemeIA AVANT envoi.
4. **Détections multiples :** Le SystemeIA effectue :
   - Analyse du langage (insultes, harcèlement)
   - Détection de coordonnées personnelles
   - Vérification du caractère éducatif

Décision : Le SystemeIA retourne "légitime" ou "inapproprié".

Si légitime :

- Le message est enregistré dans la base
- Le message est livré au destinataire
- Notification envoyée au destinataire
- Confirmation affichée à l'expéditeur

Si inapproprié :

- Le message est bloqué AVANT envoi
- Le SystemeIA enregistre l'infraction
- Message d'erreur affiché à l'expéditeur
- Compteur d'infractions incrémenté
- Le destinataire ne reçoit rien et n'est pas au courant

Diagramme de séquence :

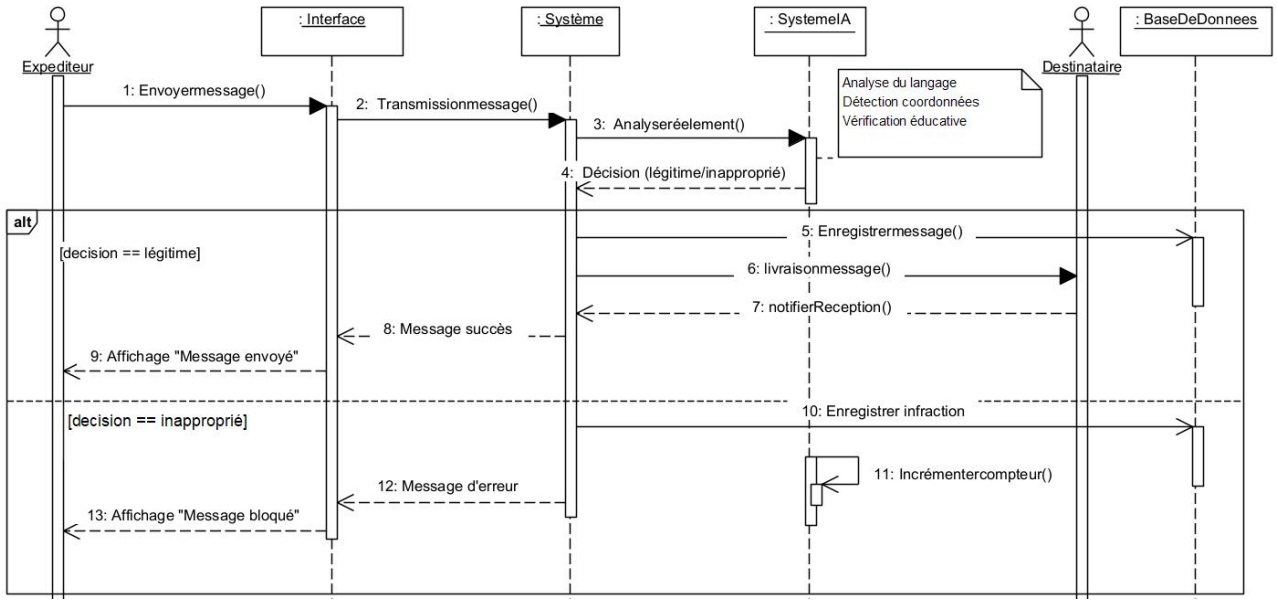


Figure 5 : Diagramme de séquence - Chat privé avec surveillance IA

Note relative à la modélisation : Bien que Visual Paradigm affiche une barre d'exécution sur les acteurs, celle-ci ne signifie pas qu'ils effectuent un traitement. Conformément aux standards UML, les acteurs sont ici des intervenants passifs ; la barre ne sert qu'à marquer leur participation à l'échange.

3- Scénario 3 : Passer un quiz adaptatif .

Contexte : Un apprenant se connecte et le système génère automatiquement un quiz personnalisé basé sur son activité récente.

Acteurs impliqués :

- **Apprenant** : L'apprenant qui va passer le quiz
- **Interface** : L'interface de quiz
- **Système** : Le système central
- **SystemeIA** : Le générateur de quiz
- **BaseDeDonnees** : Stockage de l'historique et des résultats

► Déroulement chronologique détaillé:

1. **Connexion :** L'apprenant se connecte .
2. **Récupération de l'historique :** Le système interroge la base de données pour obtenir :
   - La dernière ressource consultée (vidéo regardée, PDF ouvert, cours suivi)
   - Les 5 derniers quiz passés avec leurs scores
   - Le niveau actuel de l'apprenant (calculé à partir de ses performances)
   - Les concepts mal maîtrisés (identifiés lors des quiz précédents)
   - Le temps écoulé depuis le dernier quiz
3. **Appel de l'IA :**Le système demande au SystemeIA de générer un quiz
4. **Analyse par l'IA :** Le SystemeIA analyse les données pour déterminer :
   - Le contenu à évaluer : quel est le dernier sujet étudié ?
   - Le niveau approprié : l'apprenant est-il débutant, intermédiaire ou avancé dans ce sujet ?
   - Les concepts à renforcer : quels sont les points faibles détectés ?
   - Le nombre de questions : en fonction du temps disponible et de la complexité
5. **Génération du quiz :** Le SystemeIA crée un quiz sur mesure :
   - Sélection de 5 à 10 questions depuis une banque de questions
   - Ajustement de la difficulté (facile, moyen, difficile) selon le niveau
   - Équilibrage des types de questions (QCM, vrai/faux, questions ouvertes courtes)
   - Ordre des questions optimisé (du plus simple au plus complexe)
   - Attribution de points par question selon la difficulté
6. **Affichage du quiz :** Le système affiche le quiz généré sur le tableau de bord avec trois options:
   - "Faire le quiz maintenant"
   - "Reporter à plus tard" (le quiz restera disponible dans "Quiz en attente")
   - "Ignorer ce quiz" (le quiz sera supprimé, un nouveau sera généré à la prochaine connexion)

Si l'apprenant choisit de faire le quiz :

1. **Démarrage** : L'apprenant commence le quiz et répond aux questions
2. **Soumission** : Le système soumet le quiz

3. **Correction par l'IA** : Le SystemeIA corrige instantanément :
    1. Compare les réponses
    2. Calcule le score
    3. Génère un feedback détaillé
4. **Enregistrement** : Le système enregistre le ResultatQuiz
5. **Mise à jour** : Le système met à jour la Progression
6. **Affichage des résultats** : L'interface affiche :
    1. Score global
    2. Feedback question par question
    3. Concepts à revoir
7. **Recommandations** : Le SystemeIA génère des recommandations selon le score

► Remarque :

- **Si l'apprenant reporte le quiz :** Le quiz est sauvegardé dans une liste "Quiz en attente" accessible depuis le tableau de bord. Un rappel est envoyé après 24 heures.
- **Si l'apprenant ignore le quiz :** Le quiz est supprimé et un nouveau sera généré lors de la prochaine connexion, basé sur l'activité mise à jour.

Diagramme de séquence :

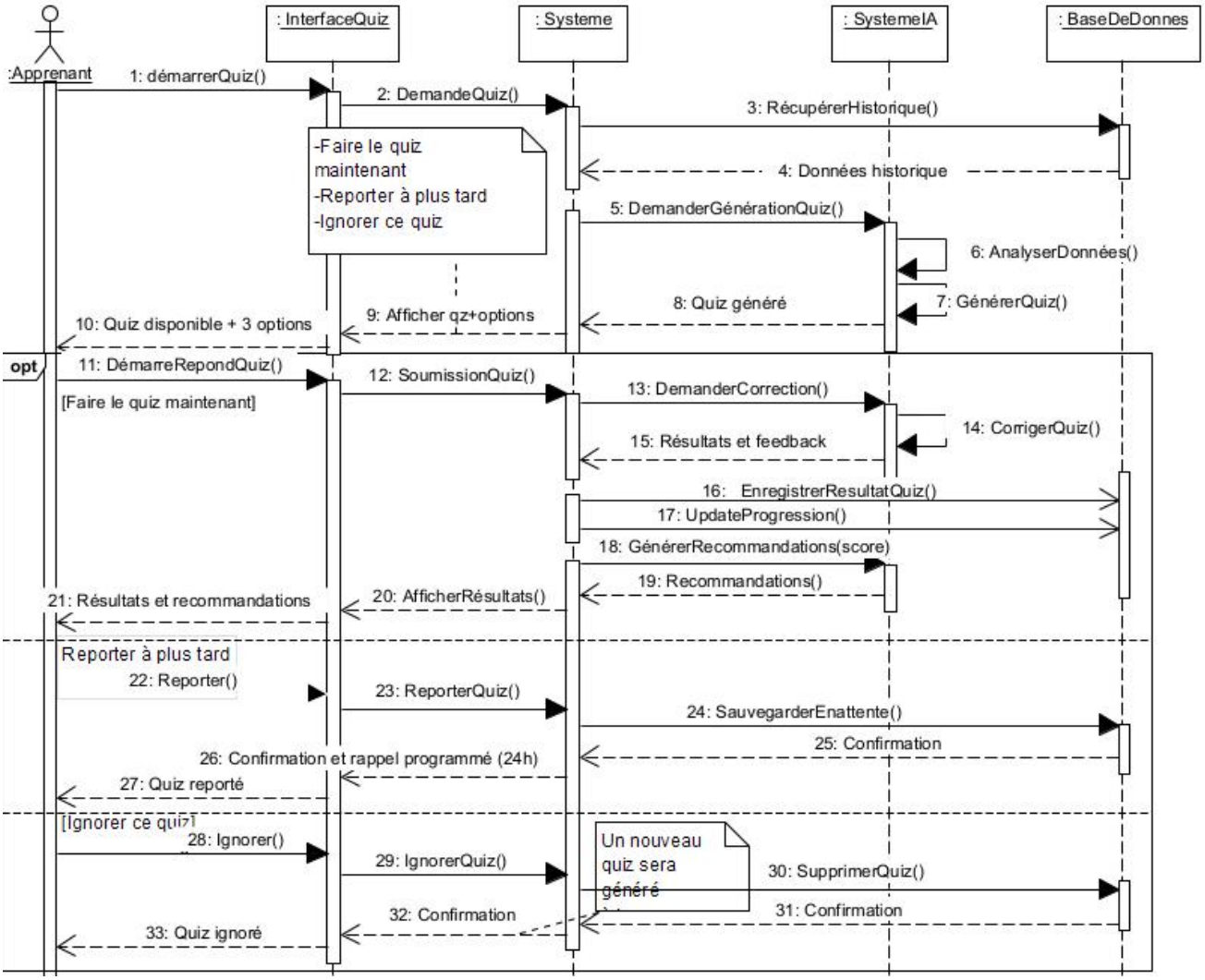


Figure 6 : Diagramme de séquence - Quiz adaptatif avec correction IA

Note relative à la modélisation : Bien que Visual Paradigm affiche une barre d'exécution sur l'acteur, celle-ci ne signifie pas qu'il effectue un traitement. Conformément aux standards UML, l'acteur est ici un intervenant passif ; la barre ne sert qu'à marquer sa participation à l'échange.

V- Diagramme de paquetages :

Le diagramme de paquetages représente l'organisation modulaire du système en regroupant les classes par domaine fonctionnel. Cette organisation facilite la maintenance et l'évolution de la plateforme.

♦ Organisation modulaire du système :

Le système est organisé en **5 paquetages principaux et d'un sous-paquetage**, chacun regroupant un ensemble cohérent de classes :

Paquetage 1 : Utilisateur	**Responsabilité :** Gestion de tous les aspects liés aux utilisateurs et à l'authentification. **Classes contenues :** Utilisateur (abstraite), Apprenant, Formateur, Administrateur **Rôle :** Paquetage fondamental utilisé par tous les autres paquetages.
Paquetage 2 : Formation	**Responsabilité :** Gestion complète des formations, de leur contenu et de leur accès. **Classes contenues :** Formation, Module, Ressource (abstraite), Video, PDF, Image, Audio, CodeAcces **Rôle :** Cœur métier de la plateforme. Définit la structure et l'organisation des contenus pédagogiques.
Paquetage 3 : Evaluation	**Responsabilité :** Gestion des évaluations, quiz et résultats. **Classes contenues :** Quiz, Question, Reponse, ResultatQuiz **Rôle :** Permet la mesure objective des connaissances acquises.
Paquetage 4 : Communication	**Responsabilité :** Gestion de toutes les interactions et échanges entre utilisateurs. **Classes contenues :** Chat, Message, Publication, Groupe, Classroom, Bibliotheque, RessourceBibliotheque **Rôle :** Gère l'aspect social et collaboratif de la plateforme.
Paquetage 5 : Groupement	**Responsabilité :** Organisation collective des utilisateurs en groupes et classrooms. C'est un sous paquetage de Communication. **Classes contenues :** Groupe, Classroom (peuvent être extraites du package Communication pour plus de clarté) **Rôle :** Facilite l'apprentissage collaboratif et permet aux formateurs de structurer leurs cours.
Paquetage 6 : Suivi	**Responsabilité :** Analyse et suivi de la progression des apprenants. **Classes contenues :** Progression, Inscription, Paiement, Recommandation **Rôle :** Fournit les données essentielles pour la personnalisation de l'apprentissage.
Paquetage 7 : IA et Securite	**Responsabilité :** Centralisation de toutes les fonctionnalités d'intelligence artificielle et Gestion de la sécurité, de la surveillance et des sanctions. **Classes contenues :** SystemeIA , Infraction, ListeSurveillance **Rôle :** Cœur innovant de la plateforme. Intervient de manière transversale sur presque tous les autres paquetages et garantit l'intégrité et la sécurité de la plateforme.

♦ Dépendances entre paquetages :

Le diagramme présente **uniquement** les dépendances essentielles pour assurer la lisibilité. Ces dépendances sont organisées selon deux patterns architecturaux principaux :

- ✓ **Pattern 1 : Dépendance universelle vers Utilisateur** Tous les paquetages dépendent du paquetage Utilisateur car l'authentification est obligatoire pour accéder à toute fonctionnalité.Cette dépendance est de type **<<import>>** car les paquetages instancient ou utilisent directement les classes Utilisateur, Apprenant, Formateur, ou Administrateur.
- ✓ **Pattern 2 : Formation comme paquetage pivot** Le paquetage Formation est utilisé par trois autres paquetages (Evaluation, Groupement, Suivi). C'est l'objet métier principal autour duquel s'organise la plateforme. Cette dépendance est de type **<<access>>** car ces paquetages consultent les contenus de formation sans modification structurelle.

Diagramme de Paquetages :

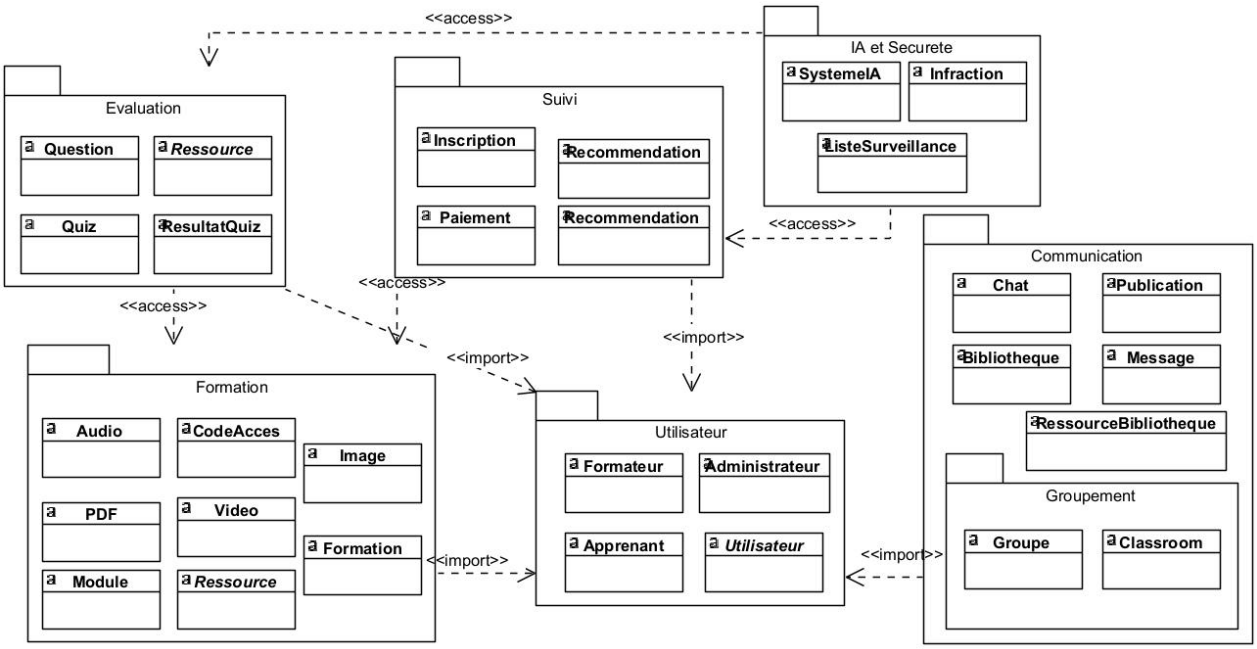


Figure 7 : Diagramme de paquetages - Architecture modulaire du système

VI- Diagramme de composents :

Le diagramme de composants représente la **structure physique** du système en montrant les parties modulaires (composants) qui le composent et leurs relations.

1- Architecture du système :

Notre plateforme est composée de **5 composants principaux** organisés en architecture 3-tiers :

Composant	Description
Interface Web	➢ **Rôle** : Gestion de l'interface utilisateur et interaction avec la platforme ➢ **Interfaces requises** : IGestionCours, IGestionQuiz, IIA
Gestion des Cours	➢ **Rôle** : Gestion complète des formations (création, modification,Consultation) ➢ **Interfaces fournies** : IGestionCours ➢ **Interfaces requises** : IStockage, IIA
Gestion des Quiz	➢ **Rôle** : Gestion des évaluations (création, traitement des quiz) ➢ **Interfaces fournies** : IGestionQuiz ➢ **Interfaces requises** : IStockage, IIA
Intelligence IA	➢ **Rôle** :Analyse des données Personnalisation des parcours et recommandations adaptatives ➢ **Interfaces fournies** : IIA ➢ **Interfaces requises** : IStockage
Base de Données	➢ **Rôle** : Stockage persistant de toutes les données de la plateforme. ➢ **Interfaces fournies** : IStockage (opérations CRUD)

	➢ **Interfaces requises** : Aucune

2- Relations entre les composants :

Les dépendances entre les composants sont représentées dans le diagramme de composants.
L'Interface Web consomme les services offerts par les composants métiers via leurs interfaces respectives (IGestionCours, IGestionQuiz, IIA).
Les composants métiers accèdent à la persistance des données uniquement par l'interface IStockage, fournie par le composant Base de Données.

Cette organisation garantit une **séparation stricte des responsabilités**, une meilleure maintenabilité du système et facilite son évolution future.

3- Diagramme de composants :

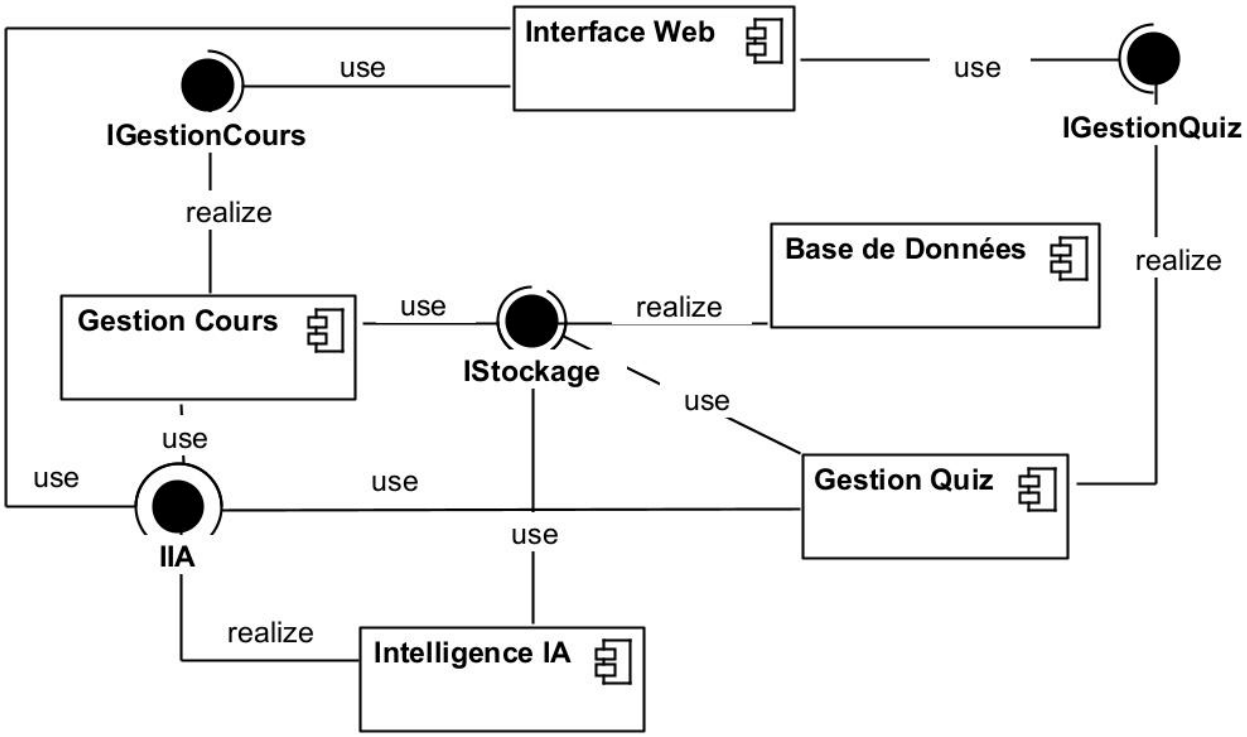


Figure 8 : Diagramme de composants - Architecture du système

VII- Diagramme d'états-transitions :

Le diagramme d'états-transitions modélise les différents états d'un objet durant son cycle de vie et les transitions entre états. Nous présentons une diagramme essentiel d'un cours durant son cycle de vie et les événements qui déclenchent le passage d'un état à un autre.

1. Description des États et Activités

Le cycle de vie est structuré en quatre phases principales :

A. Phase de Conception (Draft) :

Début : Point d'entrée initial lors de la création d'une nouvelle instance de cours.
En création (Draft) : État composite où le cours est en cours de rédaction. Il n'est pas encore visible par les apprenants.
Historique profond (H*) : Le système mémorise l'étape exacte de l'édition pour permettre au formateur de reprendre son travail là où il l'avait laissé.
Sous-état Édition : Activité interne (do / ajouter modules, uploader ressources...) représentant le travail actif du formateur.

B. Phase de Validation IA :

En attente de validation IA : État pivot où le contenu est soumis à une analyse automatique de conformité et de sécurité.
Rejetée par IA : État atteint si des problèmes sont détectés par l'algorithme. Le formateur doit utiliser l'action "Corriger le contenu" pour revenir en mode Draft.

C. Phase de Cycle de Vie Public :

Cet état composite regroupe les situations où le cours est officiellement enregistré dans le catalogue.
Publiée (Active) : Le cours est en ligne, visible, et les apprenants peuvent s'y inscrire.
Suspendue : Le cours est temporairement retiré de la vente ou de la consultation via l'action "suspendre". Il peut être remis en ligne par l'action "Réactiver".
En modification : État déclenché par l'action "modifier" depuis un cours publié. Une fois les changements effectués, l'action "Mise à jour" renvoie le cours vers une nouvelle validation IA.

D. Phase de Fin de Vie :

Archivée : État où le cours est retiré du catalogue principal mais conservé pour les archives historiques.
Supprimée : État final du système. L'objet est définitivement retiré après l'action "Supprimer".

1. Tableau des Transitions et Événements :

État Source	Événement / Action	État Cible	Description
Début	(Automatique)	En création	Initialisation du cours.
En création	**Publier**	En attente validation IA	Soumission pour contrôle qualité.
En attente IA	**Validation réussie**	Publiée (Active)	Mise en ligne officielle.
En attente IA	**Problèmes détectés**	Rejetée par IA	Échec du contrôle qualité.
Rejetée par IA	**Corriger le contenu**	En création	Retour en édition corrective.
Publiée	**modifier**	En modification	Ouverture de l'éditeur de mise à jour.
En modification	**Mise à jour**	En attente validation IA	Nouvelle validation obligatoire après modif.
Cycle Public	**archiver**	Archivée	Retrait du catalogue.

Archivée	**Supprimer**	Supprimée (Final)	Destruction définitive.

2. Le Diagramme d'états :

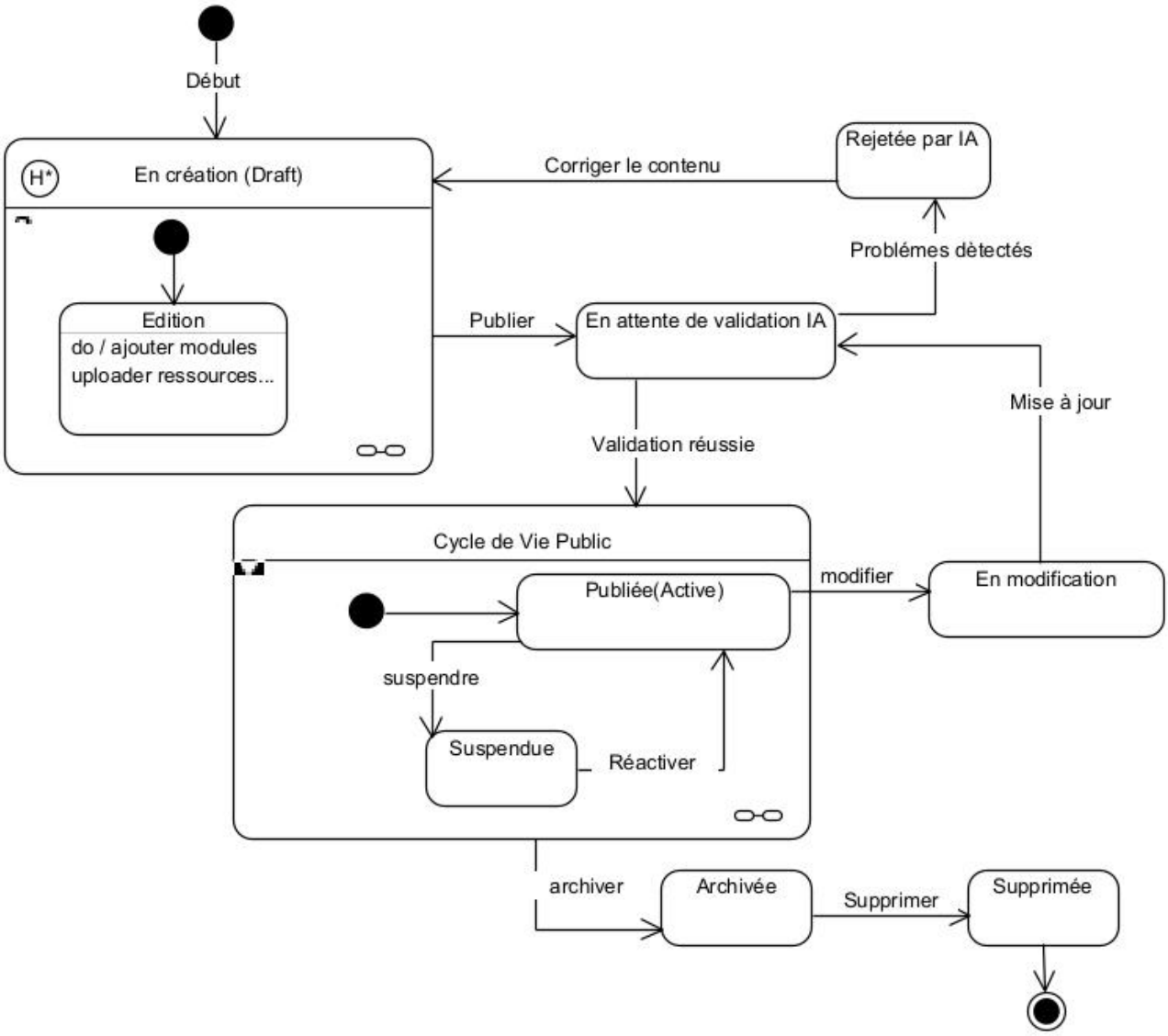


Figure 9 : Diagramme d'états-transitions

VIII- Diagramme d'activités :

Le diagramme d'activités modélise le flux de travail du système. Il représente l'enchaînement des actions, les décisions et les synchronisations nécessaires pour accomplir une tâche complexe. Nous présentons ici une processus essentiels du système.

1- Processus global d'apprentissage :

- **Contexte :** Un apprenant souhaite suivre une formation complète sur la plateforme, de l'inscription initiale jusqu'à la certification finale.
- **Acteurs impliqués :**

Acteur	Rôle
Apprenant	Utilisateur principal qui suit le processus
Système	Coordonne les différentes étapes
SystemeIA	Intervient pour personnaliser l'expérience
Formateur	Valide certaines étapes (pour formations manuelles)

➢ Déroulement du processus :

Étape 1 : Inscription et authentification	L'apprenant crée un compte ou se connecte **Décision** : authentification réussie → continuer / sinon → erreur
Étape 2 : sélection de formation	Navigation dans le catalogue avec filtres Le SystemeIA propose des recommandations personnalisées Sélection d'une formation
Étape 3 : Vérification du type d'accès	**Formation gratuite :** - Accès automatique → inscription immédiate, accès direct au contenu - Accès manuel → demande au Formateur → accepté / refusé **Formation payante :** Paiement sécurisé→Si réussi→Génération code d'accès→ Inscription validée Si échoué → Message d'erreur, possibilité de réessayer
Étape 4 : Accès au contenu et apprentissage	Accès au tableau de bord (modules, chapitres, ressources) Création objet Progression **Boucle :** Consultation ressources → enregistrement progression → notes/favoris **Activité parallèle :** Pendant la consultation, le SystemeIA analyse le comportement pour détecter les difficultés potentielles
Étape 5 : Quiz et évaluations	Quiz à la fin de chaque module (il est proposé manuel ou généré par l'IA) Correction instantanée par SystemeIA et génère un feedback détaillé **Sous-processus : Passer un quiz adaptatif** (voir diagramme de séquences) Le résultat est enregistré dans ResultatQuiz **Décision** : Score ≥ seuil → module validé / sinon → révision et repasser après 24h (Recommandation)
Étape 6 : Recommandations personnalisées	Le SystemeIA analyse en permanence la progression Suggestions de formations avancées ou ressources complémentaires
Étape 7 : Complétion de la formation	Tous modules obligatoires terminés (progression = 100%) **Décision** :Quiz final obligatoire ? Si oui : Score ≥ 70% → certification délivrée/ sinon → repasser quiz Si non → formation terminée sans certification
Étape 8 : Certification et finalisation	Génération automatique d'un certificat PDF Badge numérique ajouté au profil Propositions de formations complémentaires
Étape 9 : Évaluation de la formation (optionnel)	L'Apprenant est invité à noter la formation (1 à 5 étoiles) et Il peut laisser un commentaire (analysé par l'IA pour détecter les contenus inappropriés) Cette évaluation aide l'IA à améliorer les futures recommandations

- **Diagramme d'activités :** Ce diagramme d'activités illustre le parcours complet d'un apprenant depuis son inscription jusqu'à l'obtention d'une certification.

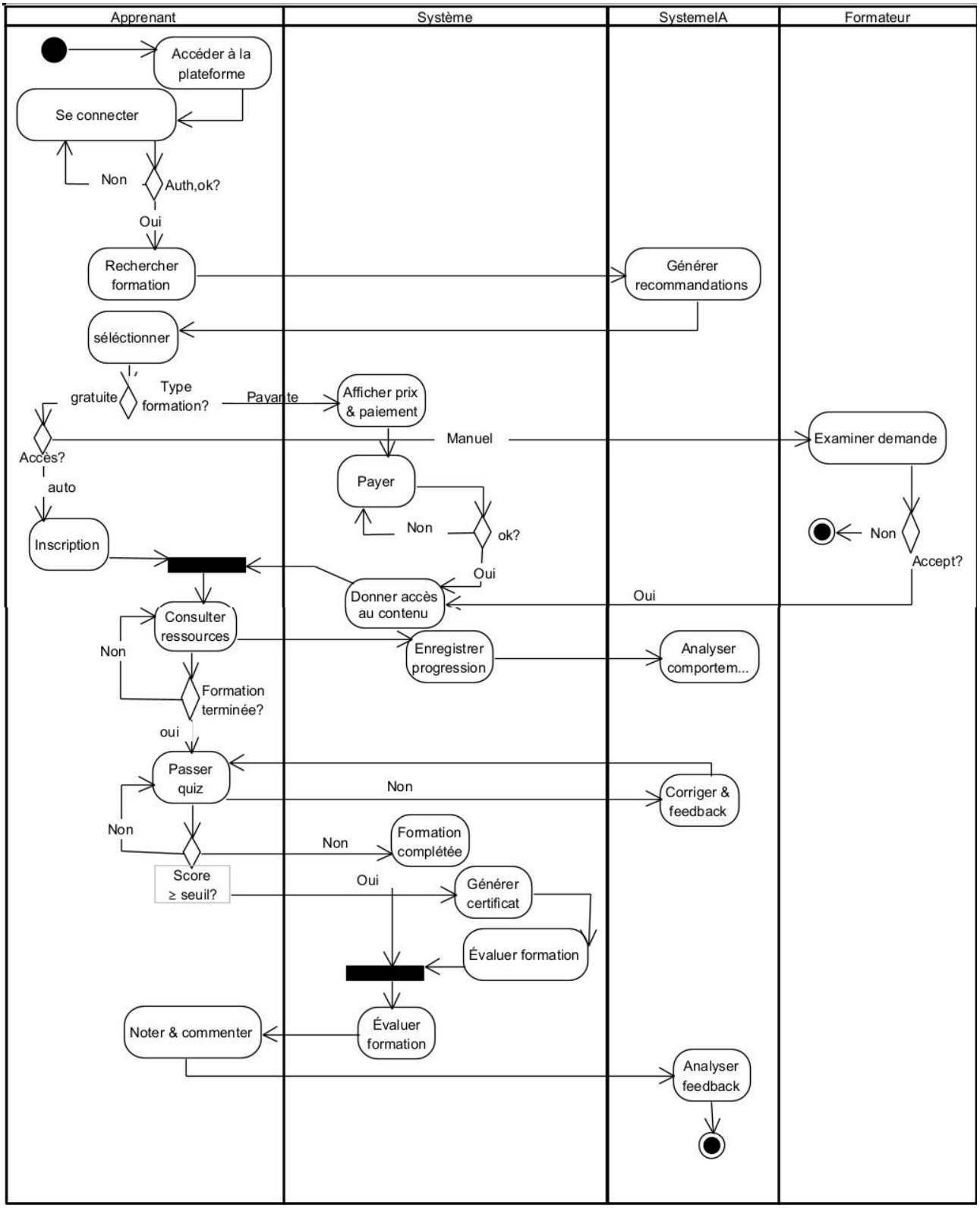


Figure 10 : Diagramme d'activités - Processus global d'apprentissage

CONCLUSION

Ce projet avait pour objectif de concevoir, à l'aide de la méthodologie **UML**, une plateforme de gestion de cours en ligne intégrant l'intelligence artificielle afin de répondre aux défis actuels de l'e-learning. Le travail réalisé s'est appuyé sur une analyse approfondie des besoins du domaine éducatif et sur une démarche de conception structurée et rigoureuse.

La conception du système a reposé sur l'élaboration de plusieurs diagrammes UML complémentaires, offrant une vision globale, cohérente et détaillée de la plateforme. Les diagrammes structurels ont permis de définir l'architecture statique du système, en identifiant les acteurs, les classes, les objets et les modules fonctionnels. Les diagrammes comportementaux ont, quant à eux, modélisé la dynamique du système à travers les scénarios clés, les cycles de vie et les processus métier fondamentaux.

L'utilisation de la méthodologie UML s'est révélée essentielle pour structurer la réflexion, assurer la cohérence de la conception et faciliter la compréhension globale du système. Les diagrammes produits constituent une documentation complète et exploitable, servant de base solide pour une éventuelle phase de développement et d'implémentation.

L'originalité de la solution proposée réside dans son approche intégrée, où la sécurité, la personnalisation de l'apprentissage et la qualité pédagogique ne sont pas traitées comme des fonctionnalités isolées, mais comme des composantes centrales soutenues par une intelligence artificielle omniprésente. Cette conception permet de répondre de manière concrète aux limites des plateformes éducatives actuelles.

En conclusion, ce travail aboutit à la conception d'un système modulaire, évolutif et maintenable, capable d'être déployé dans un contexte professionnel réel. Il ouvre également des perspectives d'évolution, notamment l'implémentation technique de la plateforme, l'intégration d'algorithmes d'IA plus avancés et l'évaluation de son impact pédagogique auprès des utilisateurs.

Bibliographie :

- ✓ Support de cours UML - Filière IIA S3- Pr. Anas EL ANSARI - 2025/2026
- ✓ CNRS (LIRIS) : Modélisation UML.
- ✓ L. Audibert (Developpez.com)
- ✓ SlidesShare.com
- ✓ Scribd.com
- ✓ Logiciel utilisé : Visual-Paradigm CE (Community Edition).